

[Need Backup]

2. Java Servlets

second in the series.

=====

A **Servlet** in Java is a server-side Java program that handles client requests and generates dynamic web content. It runs on a **Java EE (Jakarta EE) server** and is primarily used for creating web applications. We create servlet class to handle http request received from postman/browser/curl etc...

Key Features of Servlets:

1. **Handles HTTP Requests** – It processes client requests (such as form submissions) and generates responses.
2. **Runs on a Web Server** – Requires a servlet container (e.g., **Tomcat, Jetty, WildFly**) to execute.
3. **Lifecycle Managed by the Container** – The web server manages the lifecycle (creation, execution, destruction).
4. **Thread-Safe** – Each request runs in its own thread (default behavior).
5. **Platform Independent** – Written in Java, so it runs on any platform with a compatible JVM.

Servlet Life Cycle

1. **Loading & Instantiation** – The servlet class is loaded when first requested.
2. **Initialization (init() method)** – Called once to initialize resources.
3. **Handling Requests (service() method)** – This method invokes:
 - doGet() for HTTP GET requests.

- doPost() for HTTP POST requests.
4. **Destroy (destroy() method)** – Called before removing the servlet instance.

Do I need to call init() method to initialize servlet ?

ChatGPT said:

No, you **do not** need to call the init() method manually. The **Servlet Container (like Tomcat, Jetty, or WildFly)** automatically calls the init() method when the servlet is first loaded.

How Servlet Initialization Works

1. When a request is made for the first time, the **container loads the servlet class**.
2. It **creates an instance** of the servlet.
3. The container **calls the init() method** to initialize the servlet.
4. Once initialized, the servlet **remains in memory**, handling multiple requests using **separate threads**.
5. The destroy() method is called before shutting down the application or when the servlet is removed.

how do you call service() method of servlet ?

ChatGPT said:

You **do not** call the service() method manually. The **Servlet Container (e.g., Tomcat, Jetty, WildFly)** automatically calls the service() method when a request is received.

How service() Works in a Servlet

1. A **client (browser, API client, etc.) sends a request** to the servlet.
2. The **web container intercepts the request**.

3. The container **calls the service() method**, which checks the HTTP request type:
- If it's a GET request, doGet() is called.
 - If it's a POST request, doPost() is called.
 - Similarly, doPut(), doDelete(), etc., are called for respective HTTP methods.

Can You Call service() Manually?

Yes, but **it's not recommended**. If you do, it will execute without the container's proper handling.

Example of manually calling service() (not common in real projects):

```
MyServlet servlet = new MyServlet();  
servlet.service(request, response); // This bypasses the container
```

Basic Example of a Java Servlet

```
java Copy Edit  
  
import java.io.IOException;  
import java.io.PrintWriter;  
import jakarta.servlet.ServletException;  
import jakarta.servlet.http.HttpServlet;  
import jakarta.servlet.http.HttpServletRequest;  
import jakarta.servlet.http.HttpServletResponse;  
  
public class HelloServlet extends HttpServlet {  
    protected void doGet(HttpServletRequest request, HttpServletResponse response)  
        throws ServletException, IOException {  
        response.setContentType("text/html");  
        PrintWriter out = response.getWriter();  
        out.println("<h1>Hello, Servlet!</h1>");  
    }  
}
```

Steps to Deploy a Servlet

1. **Compile** the Java file.
2. **Deploy** it in a web container (like Tomcat).
3. **Configure** it in web.xml or use @WebServlet annotation.
4. **Access** it via browser: `http://localhost:8080/app/HelloServlet`.

Should You Use Servlets in Spring Boot?

Spring Boot provides **Spring MVC**, which is much more powerful than raw Servlets. Instead of Servlets, you should typically use:

- ✓ **Spring MVC Controllers** (@RestController)
- ✓ **Filters (OncePerRequestFilter)** for request processing
- ✓ **Interceptors (HandlerInterceptor)** for pre/post-processing

Would you like an example of converting this servlet into a **Spring MVC Controller**, which is the recommended approach? 🚀